Java vs Go

A Java developer's quick reference | Mathieu Néron | 2026

This cheatsheet anchors Go on constructs you already know in Java. Left column: Java. Right column: Go (1.22+). Go is intentionally minimal — the biggest mental shifts for a Java dev are: **no inheritance**, **errors as return values**, **structural interfaces**, **goroutines + channels** for concurrency.

1. Hello world & program entry

Program entry point

Java

```java
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, " + args[0]);
  }
}
// javac Hello.java && java Hello world
```

Go

```go
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println("Hello,", os.Args[1])
}
// go run hello.go world  /  go build && ./hello world
```

File / module / package layout

Java

```java
// One public class per file.
// src/main/java/com/acme/Hello.java
package com.acme;
// pom.xml or build.gradle for deps
```

Go

```go
// Each directory == one package. All .go files in
// the dir share the package namespace.
// go.mod at module root:
module github.com/acme/svc
go 1.22
require github.com/foo/bar v1.2.3
```

2. Variables, constants, type inference

Local variables

Java

```java
int x = 5;
final String s = "hi";
var list = new ArrayList<Integer>();   // Java 10+
```

Go

```go
var x int = 5
var s = "hi"          // type inferred
y := 5                // short decl, only inside funcs
const Pi = 3.14159    // typed by use site
const (
    A = iota          // 0
    B                 // 1
    C                 // 2
)
```

Zero values & visibility

Java

```java
// Fields default: 0, false, null. Locals must be init'd.
// Visibility via keywords: public/protected/private.
public class C { private int x; }
```

Go

```go
// All declared vars get the type's zero value
// (0, "", false, nil). No null-pointer surprises on init.
// Visibility: Capitalized = exported, lowercase = unexported.
type C struct { x int }   // x unexported (private)
type D struct { X int }   // X exported   (public)
```

3. Primitive types & conversions

Numeric types

Java

```java
byte b; short s; int i; long l;
float f; double d;
int n = Integer.parseInt("42");
double x = Double.parseDouble("1.5");
// Implicit widening: int -> long. Narrowing requires cast.
long L = i;    int j = (int) L;
```

Go

```go
// int8/16/32/64, uint*, float32/64, complex64/128
// `int`/`uint` are platform-width (usually 64).
import "strconv"
n, err := strconv.Atoi("42")
x, err := strconv.ParseFloat("1.5", 64)
// NO implicit conversions, even widening:
var i int32 = 5
var L int64 = int64(i)        // explicit
```

Bool, byte, rune, string

Java

```java
boolean b = true;
char c = 'A';                 // 16-bit UTF-16 unit
byte[] bs = "hi".getBytes(StandardCharsets.UTF_8);
String s = new String(bs, StandardCharsets.UTF_8);
```

Go

```go
b := true
c := 'A'                      // rune == int32 (Unicode pt)
bs := []byte("hi")            // UTF-8 bytes
s := string(bs)
// Strings are immutable byte sequences (typically UTF-8).
// for i, r := range s iterates runes, not bytes.
```

4. Strings

Interpolation & formatting

Java

```java
String name = "world";
String s1 = "Hello, " + name;
String s2 = String.format("%s = %.2f", "pi", 3.14);
// Java 15+ text block:
String body = """
    Hello,
    World
    """;
```

Go

```go
name := "world"
s1 := "Hello, " + name
s2 := fmt.Sprintf("%s = %.2f", "pi", 3.14)
// Raw string literal (backticks): keeps newlines, no escapes
body := `
    Hello,
    World
    `
```

Common operations

Java

```java
s.length(); s.toUpperCase(); s.trim();
s.startsWith("H"); s.contains("ll");
String[] parts = s.split(",");
String joined = String.join("-", parts);
```

Go

```go
import "strings"
len(s)                                  // BYTES (not runes)
strings.ToUpper(s); strings.TrimSpace(s)
strings.HasPrefix(s, "H"); strings.Contains(s, "ll")
parts := strings.Split(s, ",")
joined := strings.Join(parts, "-")
```

5. Control flow

If / else (no ternary)

Java

```java
if (x > 0) { ... }
else if (x == 0) { ... }
else { ... }

int sign = x > 0 ? 1 : (x < 0 ? -1 : 0);
```

Go

```go
if x > 0 {
    ...
} else if x == 0 {
    ...
} else { ... }

// No ternary. Idiom: a tiny helper or explicit if.
sign := 0
if x > 0 { sign = 1 } else if x < 0 { sign = -1 }

// "if with init" is a Go idiom:
if v, err := f(); err == nil { use(v) }
```

Switch

Java

```java
String label = switch (x) {
  case 1, 2  -> "small";
  case 3     -> "three";
  default    -> "other";
};
// Pattern switch (21+):
switch (s) {
  case Circle c -> ...;
  case null     -> ...;
}
```

Go

```go
switch x {
case 1, 2:
    label = "small"
case 3:
    label = "three"
default:
    label = "other"
}
// No implicit fallthrough; use `fallthrough` keyword.
// "Type switch":
switch v := s.(type) {
case Circle: ...
case nil:    ...
}
```

6. Loops (just ‘for‘)

All loops use ‘for‘

Java

```java
for (int i = 0; i < 10; i++) { ... }
while (cond) { ... }
do { ... } while (cond);
for (var x : xs) { ... }
```

Go

```go
for i := 0; i < 10; i++ { ... }   // classic
for cond { ... }                  // == while
for { ... }                       // == while(true)
for i, x := range xs { ... }      // index, value
for k, v := range m { ... }       // map
for _, x := range xs { ... }      // discard index
// 1.22+: each loop iter creates fresh `i` (no aliasing trap)
```

7. Collections & data structures

Go’s stdlib is intentionally minimal here: `slice` and `map` cover 80% of needs; the rest leans on `container/*` packages, channels, or 3rd-party libraries (e.g. `github.com/emirpasic/gods`, `google/btree`, `elliotchance/orderedmap`).

Quick reference

Java	Behavior	Go
`ArrayList<T>`	dynamic array	`[]T` (slice)
`LinkedList<T>`	doubly-linked	`container/list`
`int[] / Object[]`	fixed-size array	`[N]T` (array, value type)
`HashSet<T>`	unordered set	`map[T]struct{}`
`LinkedHashSet<T>`	insertion-ordered set	slice + `map[T]int` idiom (or `orderedmap`)
`TreeSet<T>`	sorted set	`google/btree` or `emirpasic/gods/sets/treeset`
`EnumSet`	bit-set over enum	`map[Day]struct{}` or hand-rolled bitmask
`HashMap<K,V>`	unordered map	`map[K]V` (RANGE ORDER IS RANDOMIZED)
`LinkedHashMap<K,V>`	insertion-ordered	`elliotchance/orderedmap` (PyPI-style)
`TreeMap<K,V>`	sorted map	`google/btree` or `emirpasic/gods/.../treemap`
`IdentityHashMap`	identity-keyed	`map[unsafe.Pointer]V` (rare; usually a code-smell in Go)
`EnumMap`	enum-keyed map	plain `map[Day]V`
`WeakHashMap`	weak keys	no native; not idiomatic in Go
`ArrayDeque (queue)`	FIFO	`container/list` or channel
`ArrayDeque (stack)`	LIFO	slice with `append` / pop-by-reslice
`ArrayDeque`	double-ended	`container/list`
`PriorityQueue<T>`	min-heap	`container/heap` (impl interface on a slice)
`ConcurrentHashMap`	thread-safe map	`sync.Map` (specific access patterns), or `map` + `sync.RWMutex`
`BlockingQueue`	blocking FIFO	`chan T` (buffered or unbuffered)
`DelayQueue`	expiry-ordered	`container/heap` + `time.Timer`
`List.copyOf, Map.of`	immutable views	no native; convention + lint, or freeze-on-clone
`Stream / Collectors`	lazy ops	`slices`, `maps` pkgs (1.21+); `iter` (1.23+)

Slices, arrays, maps (the natives)

Slice (dynamic) vs array (fixed)

Java

```java
int[] arr = new int[10];                  // fixed
List<Integer> xs = new ArrayList<>(List.of(1,2,3));
xs.add(4); xs.get(0); xs.remove(0); xs.size();
```

Go

```go
var arr [10]int           // fixed, value type, copied on assign
xs := []int{1, 2, 3}      // slice = (ptr, len, cap)
xs = append(xs, 4)        // may grow; reassign result!
xs[0]; len(xs); cap(xs)
xs = append(xs[:i], xs[i+1:]...)  // delete index i

// 1.21+ slices package:
import "slices"
slices.Contains(xs, 2); slices.Sort(xs); slices.Reverse(xs)
```

Slice gotcha (shared backing array)

Java

```java
// subList returns a VIEW; mutating it mutates parent.
// Convention is to copy when handing out lists.
```

Go

```go
a := []int{1,2,3,4,5}
b := a[1:3]               // {2,3}, SHARES storage
b[0] = 99                 // mutates a[1] too!
// Detach:
c := slices.Clone(b)
// or: c := make([]int, len(b)); copy(c, b)
```

Map (HashMap analog) — iteration is randomized

Java

```java
Map<String,Integer> m = new HashMap<>();
m.put("a", 1); int v = m.getOrDefault("b", 0);
m.computeIfAbsent("c", k -> 42);
for (var e : m.entrySet()) { ... }
```

Go

```go
m := map[string]int{"a": 1}
m["a"] = 1
v, ok := m["b"]                       // ok false if missing
delete(m, "a")

// Go RANDOMIZES range order on purpose. To iterate sorted:
keys := slices.Sorted(maps.Keys(m))   // 1.23+
for _, k := range keys { fmt.Println(k, m[k]) }
```

Sets (no native type — use map keys)

HashSet / LinkedHashSet / TreeSet

Java

```java
Set<String> hs = new HashSet<>();
Set<String> ls = new LinkedHashSet<>();
Set<String> ts = new TreeSet<>();
ts.first(); ts.subSet("a","z");
```

Go

```go
// Go convention: set == map with empty-struct values (0 bytes).
hs := map[string]struct{}{}
hs["a"] = struct{}{}
_, ok := hs["a"]
delete(hs, "a")

// Insertion-ordered: parallel slice tracking insertion order,
// or 3rd-party (emirpasic/gods/sets/linkedhashset).

// Sorted set: emirpasic/gods/sets/treeset, or google/btree.
```

Maps with order or sort guarantees

LinkedHashMap (insertion order)

Java

```java
Map<String,Integer> m = new LinkedHashMap<>();
m.put("b", 2); m.put("a", 1);
// Iteration: b, a (insertion order)
```

Go

```go
// Idiom 1: parallel slice of keys + map for lookup
type Ordered[K comparable, V any] struct {
    keys []K; m map[K]V
}
func (o *Ordered[K,V]) Set(k K, v V) {
    if _, ok := o.m[k]; !ok { o.keys = append(o.keys, k) }
    o.m[k] = v
}

// Idiom 2: 3rd-party
// import "github.com/elliotchance/orderedmap/v2"
// om := orderedmap.NewOrderedMap[string,int]()
```

TreeMap (sorted map)

Java

```java
TreeMap<String,Integer> tm = new TreeMap<>();
tm.put("b", 2); tm.put("a", 1);
tm.firstKey(); tm.headMap("m"); tm.floorKey("g");
```

Go

```go
// No stdlib equivalent. Pick a B-tree library:
// import "github.com/google/btree"
// type kv struct{ K string; V int }
// tr := btree.NewG[kv](32, func(a,b kv) bool { return a.K<b.K })
// tr.ReplaceOrInsert(kv{"b",2})

// For "sorted iteration of an existing map", just sort keys:
keys := slices.Sorted(maps.Keys(m))   // 1.23+
```

Queue, deque, stack, priority queue

Queue (FIFO) and Deque

Java

```java
Deque<Integer> q = new ArrayDeque<>();   // FIFO via offer/poll
q.offer(1); int head = q.poll();
// Double-ended:
q.offerFirst(0); q.offerLast(2);
q.pollFirst(); q.pollLast();

// Blocking:
BlockingQueue<Integer> bq = new ArrayBlockingQueue<>(8);
```

Go

```go
import "container/list"
l := list.New()
l.PushBack(1); l.PushFront(0)
front := l.Front().Value
l.Remove(l.Front())                  // O(1) at both ends

// Channel-based queue (THE go-idiomatic FIFO):
ch := make(chan int, 8)              // buffered, blocking
ch <- 1                              // blocks if full
v := <-ch                            // blocks if empty
close(ch); for v := range ch { ... } // drain
```

Stack (LIFO)

Java

```java
// Avoid legacy java.util.Stack.
Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.push(2);
int top = st.peek(); int v = st.pop();
```

Go

```go
// Slice as stack (idiomatic):
st := []int{}
st = append(st, 1)              // push
st = append(st, 2)
top := st[len(st)-1]            // peek
v, st := st[len(st)-1], st[:len(st)-1]   // pop
```

PriorityQueue (heap)

Java

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(3); pq.offer(1); pq.offer(2);
int min = pq.poll();          // 1
```

Go

```go
import "container/heap"

// Implement heap.Interface on a slice type:
type IntHeap []int
func (h IntHeap) Len() int           { return len(h) }
func (h IntHeap) Less(i,j int) bool  { return h[i]<h[j] }
func (h IntHeap) Swap(i,j int)       { h[i],h[j] = h[j],h[i] }
func (h *IntHeap) Push(x any)        { *h = append(*h, x.(int)) }
func (h *IntHeap) Pop() any          {
    old := *h; n := len(old); x := old[n-1]; *h = old[:n-1]
    return x
}
h := &IntHeap{3,1,2}; heap.Init(h)
heap.Push(h, 0); min := heap.Pop(h).(int)
```

Concurrent

Concurrent map and queue

Java

```java
ConcurrentHashMap<String,Long> chm = new ConcurrentHashMap<>();
chm.merge("k", 1L, Long::sum);
chm.computeIfAbsent("k", k -> heavy());

BlockingQueue<Task> q = new LinkedBlockingQueue<>();
q.put(t); Task next = q.take();
```

Go

```go
// sync.Map: optimized for read-heavy + key-stable workloads.
// For "general HashMap", prefer map + sync.RWMutex.
var m sync.Map
m.Store("k", 1); v, ok := m.Load("k")
m.LoadOrStore("k", 1)              // computeIfAbsent-ish

// Atomic tally idiom:
import "sync/atomic"
var hits atomic.Int64
hits.Add(1)

// Blocking queue == buffered channel:
q := make(chan Task, 64)
q <- t; next := <-q
```

Specialized

Counter / multiset, ring buffer, named tuple

Java

```java
Map<String,Long> tally = words.stream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
record Point(int x, int y) {}
```

Go

```go
// Counter: just a map.
counts := map[string]int{}
for _, w := range words { counts[w]++ }

// Ring buffer:
import "container/ring"
r := ring.New(5)
for i := 0; i < r.Len(); i++ { r.Value = i; r = r.Next() }

// Named tuple == struct (no anonymous tuples):
type Point struct { X, Y int }
```

Mutability / read-only

Java: `ArrayList`, `HashMap`, `HashSet` mutable. `List.of/Set.of/Map.of` return immutables. `Collections.unmodifiableX` wraps a view.
Go: `slice`, `map`, struct fields all mutable. *There is no language-level immutability.* Convention: hand out copies (`slices.Clone`, `maps.Clone`); accept read-only intent via interface methods that don't mutate, or pass a copy. Some libraries freeze-on-clone but it's not idiomatic.

8. Functions

Definition, varargs, multiple returns

Java

```java
int add(int a, int b) { return a + b; }
int sum(int... xs) {
    int t = 0; for (var x : xs) t += x; return t;
}
sum(1, 2, 3);
// No multi-return: use record or out-params.
```

Go

```go
func add(a, b int) int { return a + b }

func sum(xs ...int) int {
    t := 0
    for _, x := range xs { t += x }
    return t
}

// Multi-return is idiomatic, especially (T, error):
func divmod(x, y int) (int, int) { return x/y, x%y }
q, r := divmod(10, 3)
```

Lambdas / closures

Java

```java
Function<Integer,Integer> sq = x -> x * x;
list.forEach(System.out::println);
```

Go

```go
sq := func(x int) int { return x * x }
adder := func() func(int) int {
    sum := 0
    return func(x int) int { sum += x; return sum }
}()
adder(2); adder(3)            // 2, 5 (closure captures sum)
```

9. Structs & interfaces (NO inheritance)

Class vs struct + methods

Java

```java
public class Point {
  private final double x, y;
  public Point(double x, double y) {
    this.x = x; this.y = y;
  }
  public double dist(Point o) {
    return Math.hypot(x - o.x, y - o.y);
  }
}
```

Go

```go
type Point struct {
    X, Y float64               // exported fields
}

// Method = func with a receiver
func (p Point) Dist(o Point) float64 {
    dx, dy := p.X-o.X, p.Y-o.Y
    return math.Sqrt(dx*dx + dy*dy)
}

// Constructor: just a function, no `new` keyword needed
func NewPoint(x, y float64) Point { return Point{X: x, Y: y} }
```

Pointer vs value receiver

Java

```java
// Java: objects always passed by reference (the reference
// is by value). No equivalent distinction.
```

Go

```go
type Counter struct { n int }
func (c Counter) GetVal() int { return c.n }   // value rcv
func (c *Counter) Inc()       { c.n++ }        // ptr rcv
// Use pointer receivers when method mutates state OR
// the struct is large. Be consistent within a type.
```

Interfaces (structural, implicit)

Java

```java
public interface Stringer { String toStr(); }
public class Foo implements Stringer {
  public String toStr() { return "foo"; }
}
// Nominal: must declare `implements`.
```

Go

```go
type Stringer interface {
    String() string
}

type Foo struct{}
func (f Foo) String() string { return "foo" }
// Foo SATISFIES Stringer automatically. No "implements".
// Interfaces are STRUCTURAL: shape, not name, matches.
```

Composition replaces inheritance

Java

```java
class Animal { void breathe() { ... } }
class Dog extends Animal { void bark() { ... } }
Dog d = new Dog(); d.breathe(); d.bark();
```

Go

```go
type Animal struct{}
func (a Animal) Breathe() { ... }

type Dog struct {
    Animal                // embedded type (anonymous field)
}
func (d Dog) Bark() { ... }

d := Dog{}
d.Breathe()               // promoted from embedded Animal
d.Bark()
// No virtual dispatch. Method set "promoted" via embedding.
```

10. Generics (Go 1.18+)

Generic function & type

Java

```java
<T extends Comparable<T>> T max(T a, T b) {
  return a.compareTo(b) >= 0 ? a : b;
}
class Box<T> { final T v; Box(T v) { this.v = v; } }
```

Go

```go
func Max[T cmp.Ordered](a, b T) T {
    if a >= b { return a }
    return b
}

type Box[T any] struct { V T }
b := Box[int]{V: 5}
```

Constraints

Java

```java
// Constraint = upper bound on a class hierarchy.
<T extends Number & Comparable<T>>
T sum(List<T> xs) { ... }
```

Go

```go
// Constraints are interfaces (possibly with type sets):
type Number interface {
    ~int | ~int64 | ~float32 | ~float64
}
func Sum[T Number](xs []T) T {
    var t T
    for _, x := range xs { t += x }
    return t
}
```

11. Error handling (no exceptions)

Errors are values

Java

```java
try (var f = Files.newBufferedReader(p)) {
    return f.readLine();
} catch (IOException e) {
    log.error("read failed", e);
    throw new MyException("oops", e);
}
```

Go

```go
data, err := os.ReadFile(p)
if err != nil {
    return fmt.Errorf("read failed: %w", err)  // wrap
}
// No try/catch. Functions return (T, error).
// Caller MUST check err. Idiom: "if err != nil { return }"
```

Custom error & sentinel checks

Java

```java
public class NotFound extends RuntimeException { ... }
catch (NotFound e) { ... }
catch (RuntimeException e) { ... }
```

Go

```go
var ErrNotFound = errors.New("not found")
type ValidationError struct{ Field string }
func (e *ValidationError) Error() string { ... }

if errors.Is(err, ErrNotFound) { ... }
var ve *ValidationError
if errors.As(err, &ve) { use(ve.Field) }
```

Cleanup: defer (replaces try-with-resources)

Java

```java
try (var f = new FileInputStream(p)) {
    // f auto-closed on exit
}
```

Go

```go
f, err := os.Open(p)
if err != nil { return err }
defer f.Close()          // runs when func returns
// Multiple defers run in LIFO order. Args evaluated NOW.
```

Panic / recover (rare)

Java

```java
// throw new RuntimeException("..."); is the analog,
// but Java idiom uses exceptions widely.
```

Go

```go
panic("unrecoverable")              // nuke goroutine
defer func() {
    if r := recover(); r != nil { ... }
}()
// Reserve panic for truly unrecoverable bugs.
// Normal errors -> return error.
```

12. Modules / packages / imports

Imports

Java

```java
package com.acme.svc;
import java.util.List;
import java.util.stream.Collectors;
// Maven coords in pom.xml. Repackaged into uberjar.
```

Go

```go
package svc

import (
    "fmt"
    "time"
    "github.com/acme/lib"          // module path
    log "github.com/sirupsen/logrus" // alias
)
// go mod tidy resolves go.mod / go.sum
```

13. Concurrency (goroutines + channels)

Goroutines vs threads

Java

```java
Thread t = new Thread(() -> work());
t.start(); t.join();

ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> compute());
int v = f.get();
```

Go

```go
go work()                   // fire-and-forget goroutine
// Goroutines are cheap (~few KB stack). Spawn millions.
// No "join" - coordinate via channels or sync.WaitGroup.

var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    go func(x Item) { defer wg.Done(); process(x) }(item)
}
wg.Wait()
```

Channels

Java

```java
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(8);
q.put(42);
int v = q.take();
```

Go

```go
ch := make(chan int, 8)      // buffered (8) channel
ch <- 42                     // send
v := <-ch                    // receive
close(ch)                    // close (only sender)
for v := range ch { ... }    // drain until closed

// "Don't communicate by sharing memory; share memory by
//  communicating." - channels are the primary primitive.
```

Select (multiplex)

Java

```java
// Closest analog: CompletableFuture.anyOf(...).
CompletableFuture.anyOf(f1, f2)
    .thenAccept(System.out::println);
```

Go

```go
select {
case v := <-ch1:
    use(v)
case v := <-ch2:
    use(v)
case <-time.After(2 * time.Second):
    return errors.New("timeout")
case <-ctx.Done():
    return ctx.Err()
}
```

Context (cancel + deadline)

Java

```java
// Roughly: Future.cancel(true), interrupt flag, or
// scheduled-future timeouts. No standardized propagation.
```

Go

```go
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()
result, err := callRPC(ctx, req)
// Context flows through the call tree as the first arg.
// Idiom: func F(ctx context.Context, ...) (...).
```

Mutex / atomic

Java

```java
synchronized (lock) { ... }
ReentrantLock lk = new ReentrantLock();
AtomicInteger ai = new AtomicInteger();
ai.incrementAndGet();
```

Go

```go
var mu sync.Mutex
mu.Lock(); defer mu.Unlock()
// Run with `go test -race` to catch data races.

import "sync/atomic"
var n atomic.Int64
n.Add(1); v := n.Load()
```

14. Idiom appendix

nil / equality / common gotchas

- **nil is typed.** A `var p *T` is nil; an interface value is nil only if both its type AND value are nil. Returning `(*MyErr)(nil)` as `error` is NOT nil — common bug.
- **Equality.** `==` compares structs field-by-field, but only if all fields are comparable (no slices/maps/funcs). Slices/maps/funcs are not `==`-comparable; use `reflect.DeepEqual` or hand-roll.
- **Range copies.** `for _, x := range xs` gives `x` as a copy. Take `&xs[i]` or use index for mutation. (Pre-1.22 also: same loop variable address reused; fixed in 1.22.)
- **No constructors.** Just write a `NewT(...)` function returning `T` or `*T`.
- **Errors are values, not control flow.** Wrap with `fmt.Errorf("ctx: %w", err)`; unwrap with `errors.Is/As`. Never use panic for normal flow.
- **Goroutine leaks.** A goroutine that blocks on a never-closed channel never exits. Always have a cancellation path (`ctx` or close-the-channel).
- **Capitalization == visibility.** `Foo` exported, `foo` package-private. JSON tags use struct tags: `` `json:"foo,omitempty"` ``.
- **Tooling.** `go fmt` (or `gofmt`) is non-negotiable; `go vet`, `staticcheck`, `go test`, `go test -race`, `go build`, `go mod tidy`.